Frisbee

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

UNSW's Ultimate Frisbee club has come up with a new training drill. A group of n players stand in a circle, throwing a frisbee to each other. Initially, player 1 has the frisbee.

Each player is training for two specific throws, so they are only allowed to throw the frisbee to either of two specified distinct players (perhaps including themselves).

The coach has spotted a flaw in the drill. If some player can never receive the frisbee, they will get bored. Determine whether there is any chain of throws that eventually causes each player to receive the frisbee at least once.

Input

The first line of input consists of a single integer n ($2 \leq n \leq 200,000$), representing the number of players.

n lines follow, each describing the throws available to one player. The ith such line consists of two space-separated integers a_i and b_i ($1 \leq a_i, b_i \leq n$, $a_i \neq b_i$), representing the players who can receive the frisbee in one throw from player i.

Output

If there is any chain of throws that circulates the frisbee to all n players, display `Yes`. Otherwise, display `No`. Case will be ignored.

Scoring

For Subtask 1 (50 points):

- for all $1 \leq i \leq n$,
  - $b_i = i$.

For Subtask 2 (50 points):

- no restrictions.

Examples

<table>
<tr><th>standard input</th><th>standard output</th></tr>
<tr><td>4
3 1
4 2
4 3
2 4</td><td>Yes</td></tr>
<tr><td>4
2 1
3 2
1 3
1 4</td><td>No</td></tr>
<tr><td>6
1 3
2 6
2 5
1 4
4 5
2 6</td><td>Yes</td></tr>
<tr><td>5
2 4
2 3
2 3
4 5
4 5</td><td>No</td></tr>
</table>

Note

In the first sample case, the frisbee can reach all players: $1 - 3 - 4 - 2$.

In the second sample case, player 4 will not get the frisbee.

In the third sample case, the frisbee can reach all players: $1 - 3 - 5 - 4 - 1 - 3 - 2 - 6$.

In the fourth sample case, either players 2 and 3 or players 4 and 5 will not get the frisbee.

Note that only the first two sample cases comply with the constraints of Subtask 1.

Parade

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Sprinkles has been tasked with designing the route for Sydney's Mardi Gras parade. There are n junctions and m two-way roads connecting pairs of junctions. The route must start at junction 1 and end at junction n, following some sequence of roads. A road may be followed more than once.

There is an additional constraint for the route: it must be fabulous. Each road has been painted in one of the k colours of the rainbow. A route is said to be fabulous if and only if it includes at least one road of each colour.

Finally, Sydney is known to be hilly. Each junction has a given elevation, and travelling along a road from a lower junction to a higher junction (uphill) causes exhaustion equal to the change in elevation. Note that travelling downhill does not cause any (positive or negative) exhaustion. The total exhaustion of a route is the sum of exhaustion along each road comprising that route. The parade route should reduce exhaustion as much as possible, to ensure that the attendees can focus their energy on partying.

Help Sprinkles find the minimum exhaustion possible along a fabulous parade route.

Input

The first line of input consists of three space-separated integers n, m and k ($2 \leq n \leq 10,000$, $1 \leq m \leq 10,000$), representing the number of junctions, roads and colours respectively.

The second line of input consists of n space-separated integers $h_1, \ldots, h_n$ ($0 \leq h_i \leq 100,000$), the ith of which represents the elevation of the ith junction.

m lines follow, each describing one road. The jth such line consists of three space-separated integers a_j, b_j and c_j ($1 \leq a_j < b_j \leq n$), representing the two junctions connected by the road and the colour of the road respectively.

Output

Print the minimum total exhaustion that can be incurred on a fabulous route.

If there are no fabulous routes, print `-1`.

Scoring

For Subtask 1 (50 points):

- $k = 2$,
- $c_1 = \ldots = c_{m-1} = 1$, and
- $c_m = 2$.

For Subtask 2 (50 points):

- $2 \leq k \leq 6$.
- $1 \leq c_1, \ldots, c_m \leq k$.

Examples

standard input	standard output
5 4 2 10 20 30 10 40 1 2 1 2 3 1 3 5 1 3 4 2	50
5 3 2 10 20 30 10 40 1 2 1 3 5 1 3 4 2	-1
6 6 4 60 50 40 30 20 10 1 3 4 3 4 2 3 5 1 2 4 1 2 6 3 5 6 2	20

Note

In the first sample case, the route $1 — 2 — 3 — 4 — 3 — 5$ uses colour 1 four times and colour 2 twice, and incurs exhaustion of 10, 10, 0, 20 and 10 on its five roads.

In the second sample case, the parade can only reach junctions 1 and 2, so it is not possible to make any route, let alone a fabulous one.

In the third sample case, the route $1 — 3 — 4 — 2 — 6$ uses each colour once, and only the third road is uphill so the others contribute no exhaustion.

Note that only the first two sample cases comply with the constraints of Subtask 1.

The Skelly Gang

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

The Skelly Gang is a group of notorious bushrangers. They operate in a country with n towns and m two-way roads each connecting a pair of towns. They have identified a well-hidden spot on one particular road, where they can stake out the travellers and rob them very easily. However, this plan will only work if travellers actually use the road!

Each road has a maintenance cost, and due to budget cuts, the government has decided to only maintain the cheapest selection of roads that still enables travel between all pairs of towns. If there are multiple cheapest selections of roads, the government may choose any of them.

Travellers will not use roads that are not maintained. To ensure a steady stream of stagecoaches past their hiding spot, the Skelly Gang are planning to sabotage some other roads. Any road that they sabotage will become impossible for the government to maintain. Determine the smallest number of roads that must be sabotaged to make sure that the road with the hiding spot is maintained, no matter which cheapest selection of roads the government chooses to maintain.

Input

The first line of input consists of two space-separated integers n and m ($2 \leq n \leq 3000$, $n-1 \leq m \leq 3000$), representing the number of towns and roads respectively.

m lines follow, each describing one road. The jth such line consists of three space-separated integers a_j, b_j and c_j ($1 \leq a_j < b_j \leq n$, $1 \leq c_j \leq 100,000$), representing the two towns connected by the road and the maintenance cost of the road respectively. No two roads join the same pair of towns. The first of these m lines is the road with the hiding spot. It is guaranteed that travel is possible from any town to any other town before any sabotage takes place.

Output

Display a single integer, the smallest number of roads that must be sabotaged to ensure the maintenance of the road with the hiding spot.

Scoring

For Subtask 1 (50 points):

- your answer will be judged only on whether it is zero or nonzero, i.e. all nonzero outputs will be treated as equivalent.

For Subtask 2 (50 points):

- you must output the correct integer answer.

Examples

standard input	standard output
4 4 1 2 10 2 3 20 3 4 30 1 4 40	0
4 4 1 2 40 2 3 30 3 4 20 1 4 10	1
4 4 1 2 30 2 3 30 3 4 20 1 4 10	1
6 7 1 6 40 1 2 20 2 3 20 3 6 20 1 4 20 4 5 20 5 6 20	2

Note

In the first sample case, the government will maintain the first three roads, so no sabotage is necessary.

In the second sample case, sabotaging any of the other three roads will ensure that the first road gets maintained.

In the third sample case, the government could maintain either roads 1, 3, 4 or roads 2, 3, 4. Therefore sabotaging a road is necessary to guarantee the maintenance of road 1.

In the fourth sample case, if only one of the last six roads was sabotaged, then the government could maintain the other five.

In the second, third and fourth sample cases, any non-zero integer output will be adjudged correct for Subtask 1.